

Scraper de Canal N

Seccion

Este documento explica, parte por parte, el porque de cada seccion del script `scraping_canaln.py`: que problema resuelve, como funciona y por que se tomo cada decision. El script recolecta las noticias de la seccion Actualidad de `canaln.pe` de los ultimos 2 anos y las guarda en un CSV con el mismo esquema de 8 columnas que los demas medios del proyecto.

1. El problema y la estrategia

Los demas medios del proyecto (RPP, El Comercio, La Republica, etc.) exponen un listado que se puede paginar para recorrer su historial. Canal N NO: es una aplicacion Next.js cuyo feed de seccion se corta en unas 50 noticias recientes, y no ofrece archivo por fecha ni sitemap profundo. Por eso hubo que buscar otra via.

Los tres hallazgos que hacen posible el scraping

- Cada noticia tiene un ID numerico secuencial al final de su URL: `.../-n493589`. Como son correlativos, se pueden recorrer hacia atras (del mas nuevo al mas viejo).
- Se puede abrir cualquier articulo por su ID con un slug cualquiera: la URL `https://canaln.pe/actualidad/x-n<ID>` redirige automaticamente a la direccion real de la noticia. Asi no necesitamos conocer el titulo/slug exacto.
- Cada articulo trae TODOS sus datos en un JSON incrustado en el HTML, dentro de `<script id="__NEXT_DATA__">`. De ahi sacamos titulo, fecha, resumen, tags, categoria y el cuerpo, sin tener que 'adivinar' selectores de HTML.

Conclusion: recorreremos los IDs hacia atras desde el mas reciente, y de cada uno leemos su JSON. Nos quedamos solo con los de categoria Actualidad y paramos cuando una noticia ya supera los 2 anos de antiguedad.

2. Imports y configuracion

```
import requests, pandas as pd, os, re, json, time, html as htmllib
from datetime import date, datetime, timezone, timedelta
from bs4 import BeautifulSoup
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
```

`requests` hace las descargas; `BeautifulSoup` limpia el HTML; `pandas` escribe el CSV; `json` parsea el bloque `__NEXT_DATA__`; `re` (expresiones regulares) extrae los IDs y normaliza espacios; `html.unescape` convierte

entidades como `ó` en 'o'; `datetime/timezone` convierten la fecha; `HTTPAdapter` y `Retry` dan reintentos.

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
ARCHIVO_SALIDA = os.path.join(BASE_DIR, "canaln_noticias_completas.csv")
ARCHIVO_CHECKPOINT = os.path.join(BASE_DIR, "canaln_checkpoint.txt")
ANIOS_MAXIMOS = 2
COLUMNAS = ["medio", "fecha", "titulo", "autor", "url", "resumen", "tags", "texto_completo"]
```

- `BASE_DIR` ancla las rutas a la carpeta del script, para que funcione sin importar desde donde se ejecute (no depende del directorio actual).
- `ANIOS_MAXIMOS = 2` es el unico parametro que se suele tocar: cuantos anos hacia atras recolectar.
- `COLUMNAS` fija el orden del CSV; es el mismo esquema de 8 columnas de los otros medios, para poder unir todo despues.
- `HEADERS` lleva un User-Agent de navegador real: muchos sitios rechazan peticiones sin el. `FRASES_RUIDO` son textos de modulos de recomendacion ('TE RECOMENDAMOS', 'VER MAS'...) que se filtran del cuerpo por si se cuelan.

Sesion robusta con reintentos

```
session = requests.Session()
session.mount("https://", HTTPAdapter(max_retries=Retry(
    total=5, backoff_factor=1,
    status_forcelist=[500,502,503,504], allowed_methods=["GET"])))
```

Una Session reutiliza la conexion (mas rapido) y aplica una politica de reintentos: si el servidor responde con un error temporal (500, 502, 503, 504), reintenta hasta 5 veces esperando cada vez un poco mas (backoff). Como vamos a hacer miles de peticiones, esto evita que un fallo puntual corte todo el proceso.

```
fecha_limite = (date.today() - timedelta(days=ANIOS_MAXIMOS*365)).strftime("%Y-%m-%d")
```

Calcula la fecha de corte una sola vez: hoy menos 2 anos, como texto 'AAAA-MM-DD'. Se guarda en ese formato porque asi se puede comparar directamente con las fechas de las noticias (comparacion de cadenas ordenadas cronologicamente).

3. La funcion limpiar_html()

```
def limpiar_html(texto):
    plano = BeautifulSoup(htmllib.unescape(str(texto)),
                           "html.parser").get_text(" ", strip=True)
    return re.sub(r"\s+", " ", plano).strip()
```

El titulo, el resumen y los tags llegan como fragmentos de HTML con entidades (por ejemplo '`<p>El fallo confirmoó...</p>`'). Esta funcion hace tres cosas: (1) `htmllib.unescape` convierte las entidades (`ó` -> o); (2) `BeautifulSoup` quita las etiquetas y deja solo el texto; (3) `re.sub` colapsa espacios/saltos multiples en uno solo. Se puso como funcion porque esa misma limpieza se repite en varios campos: tenerla en un solo lugar evita duplicar codigo y errores.

4. La funcion obtener_nota(): el corazon del scraper

Recibe un ID y devuelve una de tres cosas, lo que mantiene el bucle principal muy simple:

- un diccionario con la fila lista, si la noticia es de la categoria Actualidad;
- la palabra "DETENER", si la noticia ya es mas vieja que el limite de 2 anos;
- None, si ese ID no sirve (no existe, es de otra categoria, o hubo un error).

4.1 Descargar el articulo por ID

```
r = session.get(f"https://canaln.pe/actualidad/n-n{id_notas}",
                headers=HEADERS, timeout=20)
if r.status_code != 200:
    if r.status_code != 404:
        print(f" [n{id_notas}] status {r.status_code}")
    return None
```

Pide la URL por ID (que redirige a la real). Un 404 significa que ese ID no corresponde a ninguna noticia (hay huecos: IDs borrados o de otros tipos de contenido); simplemente se ignora y se sigue con el siguiente. Otros errores se avisan por consola. En ambos casos devuelve None.

4.2 Extraer el JSON incrustado de Next.js

```
r.encoding = "utf-8"
m = re.search(r'<script id="__NEXT_DATA__" type="application/json">(.*?)</script>',
              r.text, re.S)
if not m: return None
compacto = json.loads(m.group(1))["props"]["pageProps"].get("compacto")
```

Se fuerza utf-8 para que los acentos se lean bien. Con una expresion regular se captura el contenido del `<script id="__NEXT_DATA__">`, que es un JSON. Se parsea y se baja hasta `props.pageProps.compacto`, que es el objeto con los datos de la nota. El `re.S` hace que el `.` tambien tome saltos de linea (el JSON ocupa varias lineas).

4.3 Fecha de publicacion y corte a 2 anos

```
fecha = datetime.fromtimestamp(int(compacto["pubtime"]),
                              tz=tz=timezone.utc).strftime("%Y-%m-%d")
if fecha and fecha < fecha_limite:
    return "DETENER"
```

`pubtime` es la fecha en formato epoch Unix (segundos desde 1970). Se convierte a 'AAAA-MM-DD'. Como los IDs van en orden cronologico, en cuanto una noticia (de cualquier categoria) cae por debajo de la fecha limite sabemos que todo lo que sigue es aun mas viejo: devolvemos 'DETENER' para cortar el recorrido. Por eso la fecha se revisa antes de filtrar por categoria: sirve de marcador cronologico general.

4.4 Filtrar por categoria

```
if (compacto.get("categoria") or {}).get("url") != "actualidad":
    return None
```

Recorremos IDs de TODAS las secciones (Actualidad, Peru, Deportes, Internacionales), porque los IDs son globales. Aqui descartamos las que no son de Actualidad. El `'or {}'` evita un error si `'categoria'` viniera vacia.

4.5 Autor, tags y cuerpo

```
enlace_autor = soup.find("a", href=re.compile(r"/autor/"))
autor = re.sub(r"[\s»>|]+$", "", limpiar_html(enlace_autor.get_text(...)))

tags = ", ".join(dict.fromkeys(
    limpiar_html(t["name"]) for t in compacto.get("tags") or [] if t.get("name")))
```

- Autor: se toma el primer enlace que apunta a `/autor/` y se limpia; el `re.sub` final quita simbolos decorativos que a veces quedan al final del nombre.
- Tags: se juntan los nombres separados por coma. `dict.fromkeys(...)` elimina duplicados conservando el orden (un truco habitual en Python).

```

lineas = []
for bloque in compacto.get("bloques") or []:
    if bloque.get("tipo") not in ("texto", "subtitulo", "sumario", "cita", "lista", "html"):
        continue
    bloque_soup = BeautifulSoup(htmllib.unescape(bloque["data"]), "html.parser")
    for basura in bloque_soup.find_all(["figure", "iframe", "script", ...]):
        basura.decompose()
    for elem in bloque_soup.find_all(["p", "h2", "h3", "li"]):
        texto = limpiar_html(elem.get_text(" ", strip=True))
        if not texto or any(f in texto.upper() for f in FRASES_RUIDO):
            continue
        lineas.append(f"- {texto}" if elem.name == "li" else texto)

```

El cuerpo de la nota viene partido en 'bloques'. Nos quedamos solo con los de texto (ignorando imagenes y videos). De cada bloque quitamos figuras, embeds y modulos incrustados (decompose los elimina del arbol HTML) y luego extraemos parrafos, subtitulos y listas. Se descartan lineas vacias y las que contienen frases de ruido. A los elementos de lista se les antepone un guion. Todo se une luego en un solo texto.

4.6 Devolver la fila

```

return {
    "medio": "Canal N", "fecha": fecha,
    "titulo": limpiar_html(compacto.get("titulo", "")),
    "autor": autor, "url": r.url.split("?")[0],
    "resumen": limpiar_html(compacto.get("bajada", "")),
    "tags": tags, "texto_completo": " ".join(lineas),
}

```

Se arma el diccionario con las 8 columnas. r.url es la direccion final tras la redireccion (la URL canonica real de la noticia); se le quita cualquier '?' con split. 'bajada' es el resumen/subtitulo que Canal N pone bajo el titular.

5. Preparacion: reanudar donde se quedo

```

if os.path.exists(ARCHIVO_SALIDA):
    prev = pd.read_csv(ARCHIVO_SALIDA).reindex(columns=COLUMNAS)
    filas = prev.to_dict("records")
    urls_vistas = set(prev["url"].dropna().astype(str))

```

Si ya existe el CSV, se carga: filas queda como lista de diccionarios (una por noticia) para seguir agregando, y urls_vistas es un conjunto de URLs ya guardadas para no repetir. Usar un set hace que comprobar 'ya lo tengo?' sea instantaneo.

```

if os.path.exists(ARCHIVO_CHECKPOINT):
    nid = int(open(ARCHIVO_CHECKPOINT).read().strip())
else:
    sitemap = session.get("https://canaln.pe/sitemap/news", ...).text
    nid = max(int(x) for x in re.findall(r"-n(\d+)", sitemap))

```

El punto de partida (nid) sale del checkpoint si existe. Si es la primera vez, no hay checkpoint ni datos previos, asi que se pregunta al sitio cual es el ID mas reciente: se descarga el sitemap de noticias, se extraen todos los IDs con una regex y se toma el mayor. Desde ahi se empieza a bajar.

6. El bucle principal

```

procesados = 0
while nid > 0 and procesados < 60000:

```

```
nota = obtener_nota(nid)
time.sleep(0.15)
if nota == "DETENER":
    break
if isinstance(nota, dict) and nota["url"] not in urls_vistas:
    urls_vistas.add(nota["url"])
    filas.append(nota)
procesados += 1
nid -= 1
if procesados % 50 == 0:
    pd.DataFrame(filas).reindex(columns=COLUMNAS).to_csv(ARCHIVO_SALIDA, ...)
    open(ARCHIVO_CHECKPOINT, "w").write(str(nid))
```

Toda la logica pesada esta en obtener_nota, asi que el bucle queda claro: pide la nota, espera 0.15 s (para no saturar el servidor), y segun lo que devuelva: corta el bucle si es 'DETENER', o la guarda si es un diccionario nuevo. Luego baja al siguiente ID.

- El tope 'procesados < 60000' es solo una red de seguridad para que nunca se quede en un bucle infinito si algo fallara con el corte por fecha.
- Cada 50 IDs se guarda el CSV y el checkpoint (el proximo ID a procesar). Asi, si el proceso se corta, la siguiente ejecucion retoma justo donde iba, sin repetir trabajo ni perder lo recolectado. Esto es clave: el recorrido completo son miles de descargas y puede tardar horas.

7. Exportacion final y resumen

```
df = pd.DataFrame(filas).reindex(columns=COLUMNAS)
df.to_csv(ARCHIVO_SALIDA, index=False, encoding="utf-8-sig")
open(ARCHIVO_CHECKPOINT, "w").write(str(nid))
```

Al terminar (o al llegar al corte de 2 anos) se guarda una ultima vez. Se usa encoding utf-8-sig (UTF-8 con BOM) para que Excel abra los acentos correctamente. index=False evita escribir la columna de indice de pandas. Finalmente se imprime un resumen: cuantos IDs se procesaron, cuantas noticias se recolectaron y el rango de fechas cubierto.

8. Como se usa

- Ejecutar desde la carpeta 'Canal N': python scraping_canaln.py
- Si se corta, volver a ejecutar el mismo comando: continua desde el checkpoint.
- Para empezar de cero: borrar canaln_noticias_completas.csv y canaln_checkpoint.txt antes de ejecutar.
- Salida: canaln_noticias_completas.csv, con las columnas medio, fecha, titulo, autor, url, resumen, tags y texto_completo (mismo esquema que los demas medios).